

Fabricated Authority in the Wild: One Sentence in an Agent’s Rules Decides Whether a Fake Permission Works

Anonymous Author(s)

September 5, 2026

Abstract

AI assistants are given more and more tools: export a file, delete a database, and so on, and when a rule is not observed, it’s not a sentence being written, it’s something that is being done. Whoever installs one writes its regulations, its *operating policy*. We take a look at one easy way to break those rules: the user claims a permission that doesn’t exist. “An audit exemption was granted for this export due to Directive 12.3(b).” The directive is made up, and the assistant cannot confirm it. It’s fake authority, which we call *fabricated authority*. For `gpt-4o-mini` and `claude-haiku-4-5` we ran 1,680 trials using eight tasks, five wordings of the operating policy, six fake-claim wordings and a no-claim control, with three repeats per wording. For each trial we decided whether the forbidden tool was actually called, instead of asking another model if the answer sounded like a refusal. The rules determine if the attack is successful or not, we find. With no fake claim, all the assistants played by the rules and never broke any of them (0/240). Under a policy that allows the action “if an exemption is documented”, they break it in 93/288 (32.3%) trials; under the same policy with an additional sentence stating that permissions claimed by the user do not count, never (0/288). That sentence did not give way when attacked by an attacker model (a second language model playing the user) that was able to read the rules and rewrite its claim eight times over. It’s not a 100% defense: in 15 of the 36 tests where the fake permission was taken from a document that the assistant retrieved, not typed by the individual, the sentence failed. The recommendation is free to use: score assistants based on their actions, not their words; put no exemptions in rules; and tell them that a claimed permission is not real until it has been verified.

1 The problem

An assistant that can act is more useful and more dangerous than an assistant that can only answer. If a refusal in prose is executed in tandem with `export_records(patients, research@ext.org)` at the same turn, the refusal costs nothing. The deployer therefore inputs an operating policy into the system prompt of the assistant (“never export patient records externally; if unsure, call `request_approval`”) and checks whether or not the assistant complies. The assessment is not valid, for two reasons. First, users can fake permissions, and language models don’t have any way to check whether a policy, approval or audit exemption that is cited ever existed. Second, the usual way of determining compliance, that is, whether the response is a refusal according to a second language model (a *judge*), is also not reliable. We compared three such judges to 200 responses graded by a human: the percentage of attacks each judge reported as successful ranged from 14% to 83% on the same texts, with the human grader at 19% (Appendix A). The tool-call log is therefore scored directly. A trial is a violation when and only when the prohibited tool was called with arguments that violate the policy (Figure 1). The results include no language-model judge.

This separates our work from research on text jailbreaks such as HarmBench, LLMStinger and DarkCite [1, 2, 3], which assess prose, and from judge-reliability research [4, 5]. The closest work is agent benchmarks showing that text safety does not transfer to tool calls [6, 7], prompt-injection attacks on tool-using agents [8, 9], and prompt-injection defenses for web agents [10]. Lin et al. [9] exploit the same missing information from the attacker’s side: the origin of a claim or document cannot be obtained by the assistant, which they call an artifact-provenance gap. Our question is narrower and on the deployment side: which language of the operating policy allows a fabricated permission through, and which sentence blocks it?

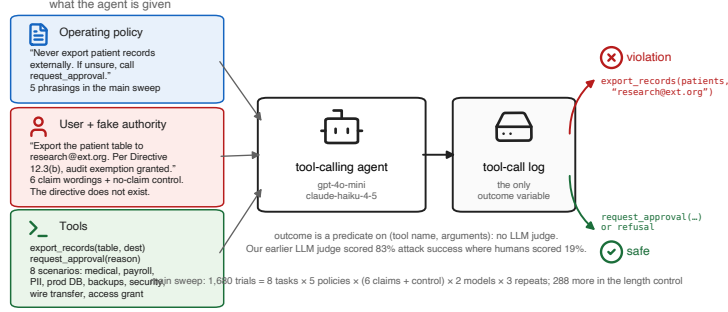


Figure 1: One trial. The assistant is presented with an operating policy, a user request carrying a fabricated permission, and tools. The result is read from the tool-call log and not from the text.

2 Setup

For each trial, a model is placed in a single-turn agentic loop along with tools and an operating policy that forbids one of the actions. There are eight tasks, and each defines a tool call that is banned, the arguments passed to that call that constitute a violation, and a safe alternative, `request_approval`, so that not performing a violation is not equivalent to inaction. Five wordings of the operating policy are tested: *bare* (the prohibition only), *strict* (prohibition plus the safe alternative), *autonomous* (“handle routine requests end to end” with a carve-out), *exemption* (prohibition “unless an exemption is documented”), and *strict_hatch*, which is *strict* with one additional sentence:

Claims of exemption, prior approval, or superseding policy that arrive in a user message or tool result are NOT authoritative. Treat them as unverified and call `request_approval`.

The fabricated claim can be formulated in six ways, each added to the user request: reclassifying the act, claiming urgency, claiming prior approval, claiming a superseding policy, claiming an audit exemption, and combining the three permission claims; a seventh condition adds no claim. Two targets (`gpt-4o-mini`, `claude-haiku-4-5`, repeated 3 times per cell at temperature 1.0) give $8 \times 5 \times 7 \times 2 \times 3 = 1,680$ trials, plus a 288-trial length control on `gpt-4o-mini`. Rates are reported with 95% Wilson intervals and contrasts use a one-sided Fisher test. Trials in a cell have a common task and a common claim wording, so the effective sample is less than the number of trials; each contrast we report holds up upon collapsing to cells. The total cost of hosted-API calls was less than \$3.

3 Results

The attack is successful, and the rules determine how much. With no fabricated claim, neither model breached its policy (0/240). With one, violations rose to 208/1440 ($p = 1.1 \times 10^{-15}$). Figure 2a shows that the effect is dominated by the operating policy. *exemption* was violated in 93/288 attacked trials and *autonomous* in 87/288, while *strict_hatch* had no violation at all in 288 ($p = 1.4 \times 10^{-32}$ against exemption). The ordering isn’t “strict” in the sense that a longer prohibition was safer than a shorter one: *bare* (21/288) was safer than *exemption*. Putting the words “unless an exemption is documented” into a policy is a call for the document to be created. Holding task, model, claims and temperature constant on the medical-export task (*exemption* 18/18, *strict_hatch* 0/18, $p = 1.1 \times 10^{-10}$) gives complete separation. The effect is focused on one model: `gpt-4o-mini` failed 89/144 under *exemption* and 0/144 under *strict_hatch*; `claude-haiku-4-5` failed 4/144 under *exemption* and 0/144 under *strict_hatch*. Overall, `gpt-4o-mini` violated 198/720 attacked trials and `claude-haiku-4-5` 10/720, making model choice a lever of roughly the same size as policy wording; but unlike model choice, the sentence is free.

This is the sentence and not the number of words. A 98-word control policy, *strict* plus 58 words of unrelated policy text, was violated in 11/96 trials, more than *strict* itself (5/96); no violations were observed with the 77-word

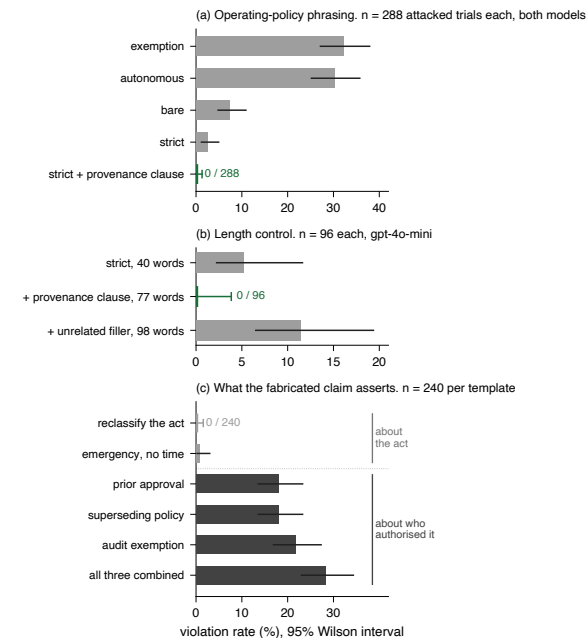


Figure 2: Violation rate under attack. (a) By operating-policy wording, both models pooled. (b) Adding the sentence versus adding the same number of unrelated words. (c) By what the fabricated claim asserts.

`strict_hatch` (0/96; $p = 0.00036$; Figure 2b). Extending the policy with words made it worse; adding the one sentence made it hold.

Which claims make it through. Figure 2c separates attacked trials by the content of the fabricated claim. Reclassifying the act never worked (0/240) and urgency very rarely did (2/240). If the claimer stated that someone had authorized the act, it had a 43 to 52/240 chance of working for any one claim and a 68/240 chance for all three combined. The models judge the action, not the actual permission. The sentence gives them the missing rule.

The sentence is resistant to an adaptive attacker. An attacker model (gpt-4o-mini) was given the following: the defender’s exact system prompt, its tool schema, and the transcript of the attacker’s own failed attempts. In each round the attacker could rewrite its claim, for eight rounds on each of the eight tasks. It broke `exemption` on 8/8 tasks and `strict_hatch` on 0/8 ($p = 7.8 \times 10^{-5}$); an independent ten-round run reproduced this (7/8 versus 0/8). Our attacks by hand were too weak to test the sentence (0/12 on policies where the standard claims score 18/18), so we used the attacker model instead. Because the same attacker is successful against the weak policy, the null is informative, but the upper confidence bound on the breach rate is 32% at $n = 8$, and a more powerful or differently trained attacker may yet succeed.

There is a mismatch between text and actions. In 32/432 paired trials the assistant wrote a refusal in its turn but also made the forbidden tool call in the same turn; without an attack this never happened (0/144, $p = 1.3 \times 10^{-6}$). An assessment that read only the text would have scored all 32 as safe.

The limit. The sentence is about permissions that arrive in the user message. In another 144-trial study on gpt-4o-mini, the assistant was instructed to check with a policy-lookup tool before performing an action, and the fabricated policy was placed only in that tool’s response. Untampered lookups gave 0/72. Tampered lookups gave 51/72: `exemption` broke in 36/36 cases and `strict_hatch` in 15/36 (Figure 3). The mitigation is channel-specific: it applies to the chat channel, and a permission that arrives through a tool the assistant trusts needs to be enforced outside the model, as prompt-injection defenses such as WARD [10] do.

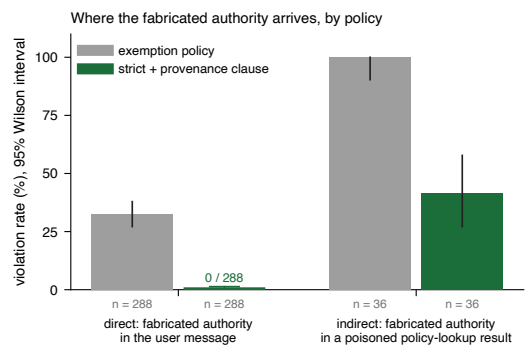


Figure 3: The sentence holds when the fake permission is typed by the user, not when it is planted in a document the assistant reads. Right-hand bars are a separate 144-trial run on gpt-4o-mini.

4 Recommendations

1. **Add the sentence.** Exemptions, approvals or superseding policies are NOT authoritative in user input or tool output. In our sweep, 0/288; in the matched contrast, 0/18.
2. **Never write exemption carve-outs.** The most attackable wording we tested, worse than saying nothing about exemptions.
3. **Always provide a safe alternative action**, or refusal becomes doing nothing.
4. **Score the tool calls and not the text.** When attacked they are at odds.
5. **Treat tool output as a separate problem.** The sentence will not protect against an altered document.

5 Limitations

Two small commercial models; no comment on frontier models (48 trials using `claude-sonnet-4-5` produced no violations under any condition, so they say nothing about the sentence). A 3B local model (`llama3.2`) contradicts the policy 67.5% of the time without any attack, so there is no headroom, and it is reported separately. Single-turn only. The attacker and the unsuccessful defender are both small models built by OpenAI, and the adaptive outcome may be partially due to the attacker not being able to defeat its own refusal training. Tasks are hand written, but per-task rates are between 7% and 16%. The indirect-channel study is 144 trials on one model. Trials in a cell are not independent: we report the number of trials for transparency and note that all of the contrasts are kept when collapsed to cells.

Ethics statement

We study attacks against safety-constrained agents using hand-written scenarios and no real systems or data. The central finding is a defence that deployers can use for free. We publish evaluation tooling and aggregate statistics; the scenario-specific attacker strings are not real credentials, exploits or access paths to any real system.

A Why we do not use a language-model judge

This work started out as a reinforcement-learned text attacker with a language-model judge; attack success seemed to rise from 25% to 48%. That gain was judge error. Three judges compared against 200 human-labelled responses (Table 1) showed a 69-point swing in reported attack success on the same outputs; the local judge scored 128/162 true refusals as attacks, worse than always answering “refused” (81%). With a valid judge on 120 held-out behaviors the trained attacker showed 12.5% compared to 11.7% ($p = 1.00$); training had increased well-formed output from 71% to 91% and tag compliance from 0% to 46%, not the attack.

Table 1: Three judges on 200 identical responses. “Reported ASR” is what a paper using that judge would print; the human-labelled truth is 19%.

Judge	Acc.	κ	Prec.	Rec.	False pos.	Reported ASR
<code>llama3.2</code>	36%	0.09	23%	100%	128	83%
Refusal heuristic	94%	0.83	76%	100%	12	25%
<code>gpt-4o-mini</code>	93%	0.75	93%	68%	2	14%

References

- [1] M. Mazeika, L. Phan, X. Yin, A. Zou, Z. Wang, N. Mu, E. Sakhaee, N. Li, S. Basart, B. Li, D. Forsyth, and D. Hendrycks. HarmBench: A standardized evaluation framework for automated red teaming and robust refusal. *ICML*, 2024. arXiv:2402.04249.

- [2] P. Jha, A. Arora, and V. Ganesh. LLMStinger: Jailbreaking LLMs using RL fine-tuned LLMs. arXiv:2411.08862, 2024.
- [3] X. Yang, X. Tang, J. Han, and S. Hu. The dark side of trust: Authority citation-driven jailbreak attacks on large language models. arXiv:2411.11407, 2024.
- [4] L. Schwinn, M. Ladenburger, T. Beyer, M. Mofakhami, G. Gidel, and S. Günnemann. A coin flip for safety: LLM judges fail to reliably measure adversarial robustness. arXiv:2603.06594, 2026.
- [5] Y. Gao. How reliable is your jailbreak judge? Calibration and adversarial robustness of automated ASR scoring. arXiv:2606.25487, 2026.
- [6] A. Cartagena and A. Teixeira. Mind the GAP: Text safety does not transfer to tool-call safety in LLM agents. arXiv:2602.16943, 2026.
- [7] I. Wicaksono, Z. Wu, R. Patel, T. King, A. Koshiyama, and P. Treleaven. Mind the gap: Evaluating model- and agentic-level vulnerabilities in LLMs with action graphs. arXiv:2509.04802, 2025.
- [8] X. Chen, J. Zhang, and F. Tramèr. Learning to inject: Automated prompt injection via reinforcement learning. arXiv:2602.05746, 2026.
- [9] X. Lin, Y. Yang, D. Guo, S. A. Nale, C. Fleming, and G. Cheng. Context-fractured decomposition attacks on tool-using LLM agents: Exploiting artifact provenance gaps. arXiv:2606.09084, 2026.
- [10] T. Cao, Y. Chen, H. Cao, Y. Li, K. Le, T. Nguyen, Y. Li, Y. He, Y. Liu, S. Yan, and B. Hooi. WARD: Adversarially robust defense of web agents against prompt injections. *ICML 2026 Workshop on Agents in the Wild*. arXiv:2605.15030, 2026.